R301

Java Persistance API

Persistance

La **persistance des données** désigne la capacité d'un système informatique à conserver et maintenir des informations de manière durable, même après l'arrêt d'une application, une panne de courant ou l'extinction complète de la machine :

- **SGBD relationnel**
- SGBD NoSQL
- Fichiers : json, csv, …
- Sérialisation et fichier

Persistance en POO

Deux mondes :

- Les objets qui exposent les méthodes et cache les attributs
- Les enregistrements qui exposent les données et n’offrent pas de méthodes

=> Comment rendre nos objets persistant dans un SGBD-R ?

JDBC et JPA

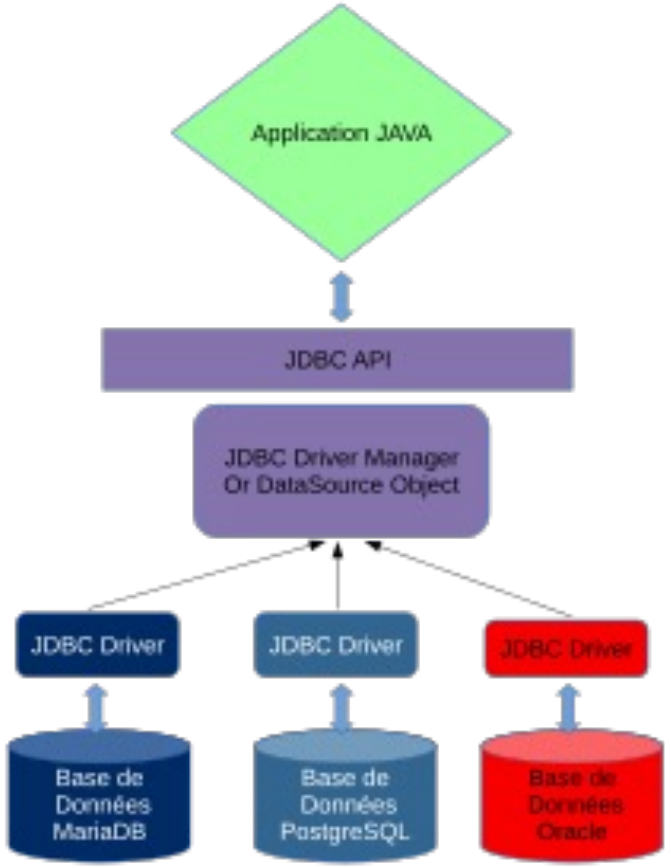
Java Database Connectivity

- Bas niveau
- Performant
- Verbeux
- Portabilité moyenne (elle peut parfois dépendre du SQL)

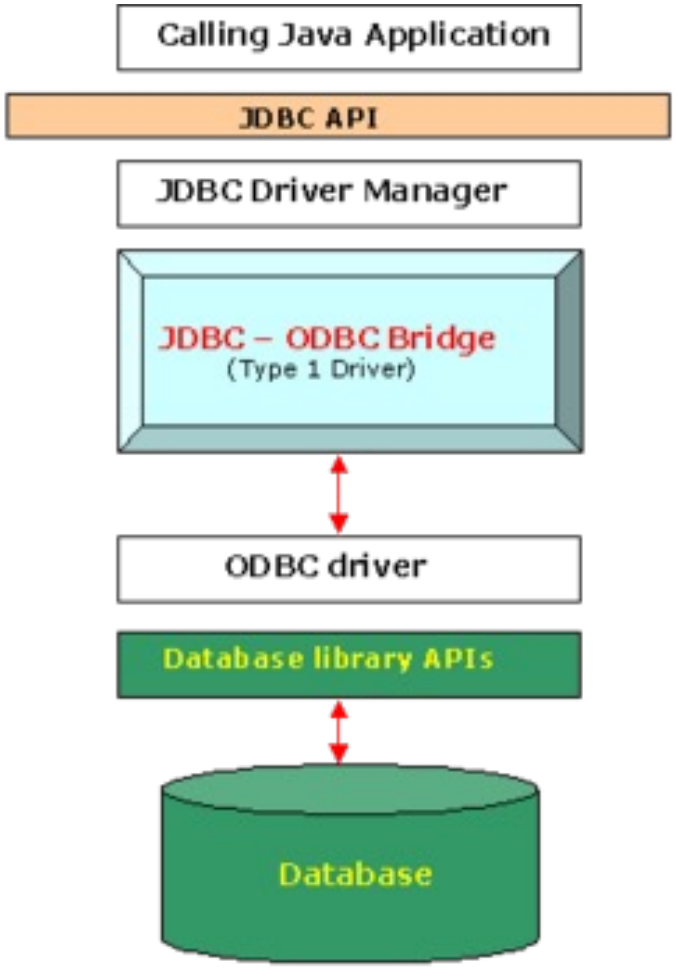
Java Persistence API

- Haut niveau
- Peut-être plus lent
- Concis
- Portabilité bonne

Approche classique : JDBC



wikipedia



Approche classique : JDBC

- API dans Java.sql et javax.sql : DriverManager, Connection, Statement, ResultSet

- Pilotes (drivers) qui traduisent les appels java en protocole propriétaires

import java.sql.Connection; import java.sql.DriverManager; import java.sql.ResultSet; import java.sql.Statement; import java.sql.SQLException;

Charger les drivers :

runtimeOnly 'com.mysql:mysql-connector-j:8.3.0’

Une URL de connexion

jdbc:<sous-protocole>:<syntaxe_propre_au_moteur_de_base_de_données>

Approche classique : JDBC

```
import java.sql.*;  // Import necessary JDBC classes

public class Demo {
    public static void main(String[] args) {

        // Define customer details to be inserted into the database
        int cust_id = 458;
        String cust_name = "Janija";
        String cust_nic = "2000256595";
        String customer_contact = "0771758937";
        String cust_email = "example_07@gmail.com";

        // Database connection details
        String url = "jdbc:mysql://localhost:3306/aloka_p";  // Database URL with schema name
        String username = "root";  // Database username
        String pasword = "Dinesh@12345";  // Database password
```

Approche classique : JDBC

```
// SQL INSERT statement with placeholders for parameters
        String sql = "insert into customer values (?, ?, ?, ?, ?)";

        try {
            // Step 1: Load the MySQL JDBC driver
            Class.forName("com.mysql.cj.jdbc.Driver");

            // Step 2: Establish the connection to the database
            Connection con = DriverManager.getConnection(url, username,
pasword);
```

Approche classique : JDBC

```
// Step 3: Create a PreparedStatement object with the SQL query
        PreparedStatement stmt = con.prepareStatement(sql);

// Step 4: Set the values for the placeholders (indexed starting from 1)
        stmt.setInt(1, cust_id);  // Set customer ID
        stmt.setString(2, cust_name);  // Set customer name
        stmt.setString(3, cust_nic);  // Set customer NIC
        stmt.setString(4, customer_contact);  // Set customer contact number
        stmt.setString(5, cust_email);  // Set customer email

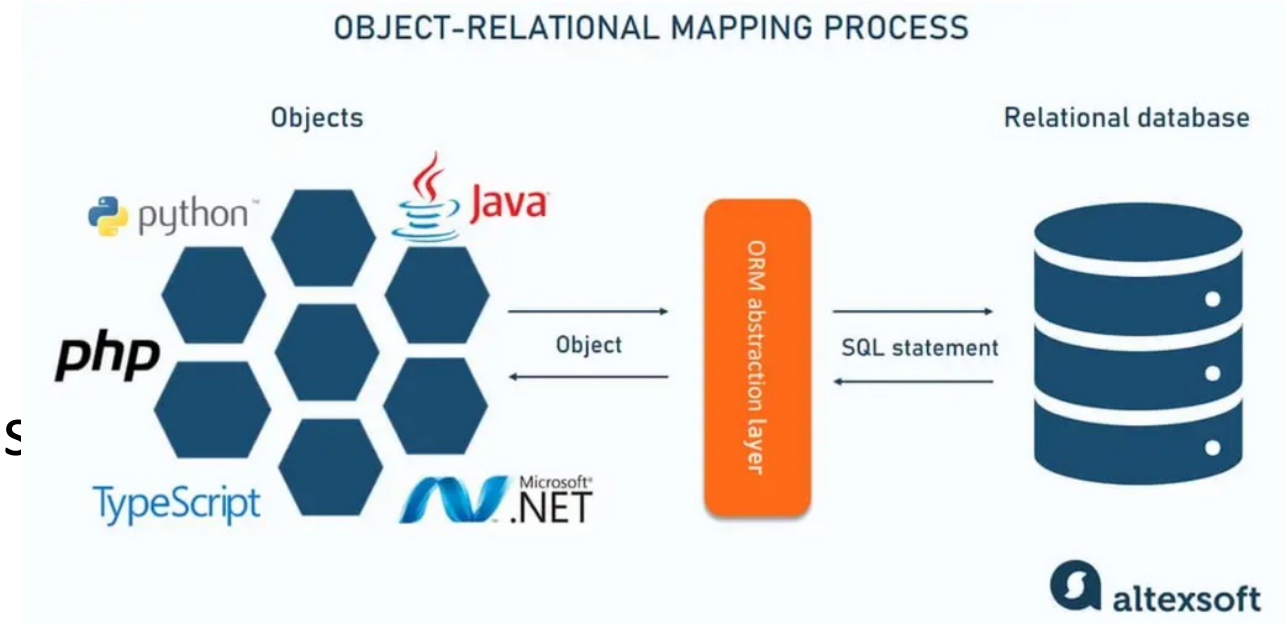
        // Step 5: Execute the SQL update to insert data
        stmt.executeUpdate();
```

Approche classique : JDBC

```
// Step 6: Close the connection to release resources
            con.close();
        } catch (Exception e) {
            // Handle exceptions by printing the error message
            System.out.println("Error: " + e.getMessage());
        }
    }
}
```

Approche ORM

Object-Relational Mapping : est un type de programme informatique qui se place en interface entre un programme applicatif et une base de données relationnelle pour simuler une base de données orientée objet.



https://www.altexsoft.com/blog/orm-object-relational-mapping/

Approche ORM

JAVA :

- Java Persistence API
- Spring Data
- Java Data Object

PHP :

- Doctrine

Python :

- SQLAlchemy

…

JPA : présentation

Java Persitence API ou Jakarta Persitence API

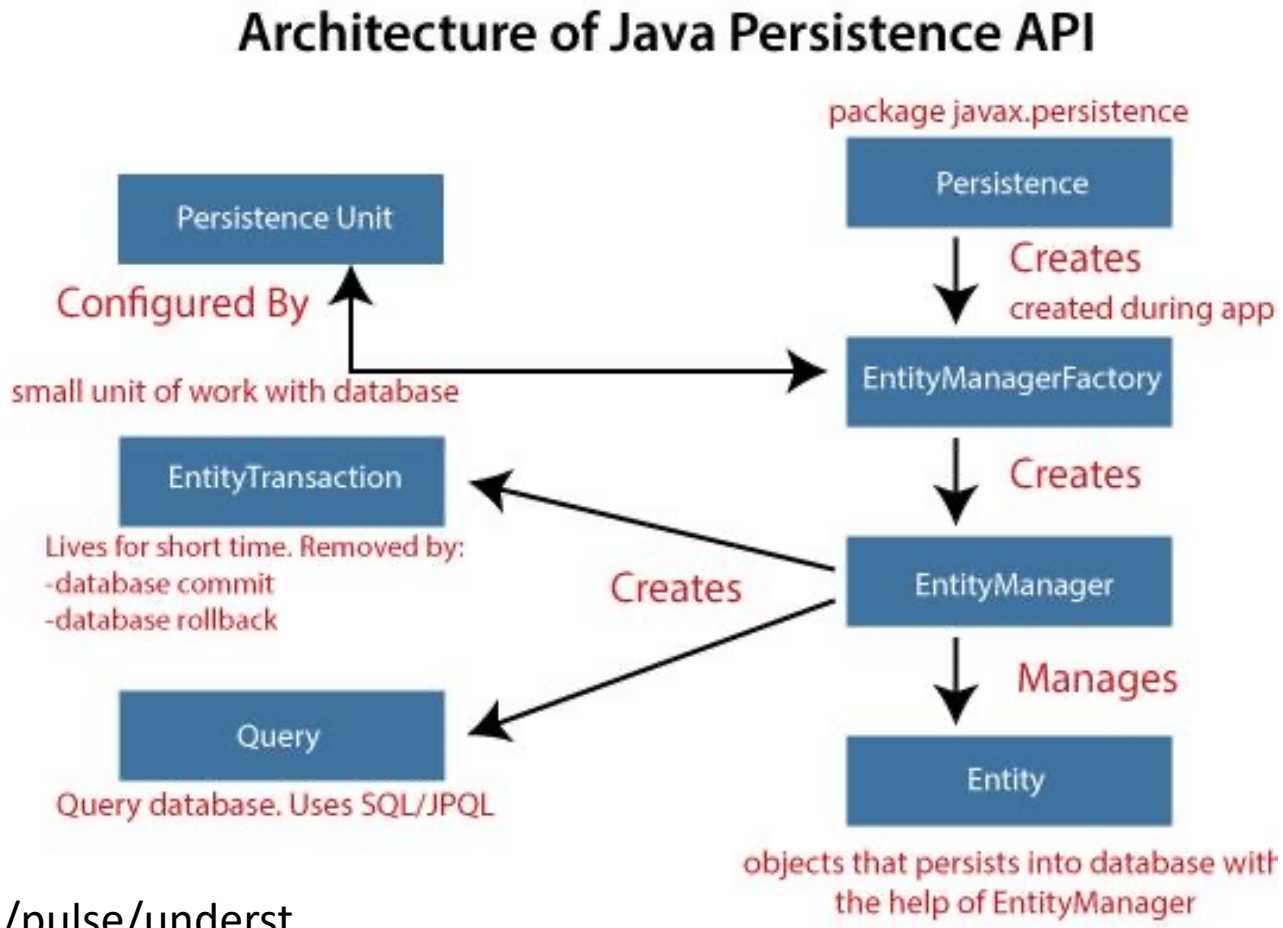
Spécification JAVA

Implémentations :
Hibernate, Toplink, EclipseLink, Apache OpenJPA, etc

3 parties :

- Les entités et les annotations
- Une API de manipulation des données
- Un langage de requête Objet

JPA : architecture



https://www.linkedin.com/pulse/understanding-jpa-java-comprehensive-guide-jakarta-api-tuan-liam--thzjc/

JPA

A inclure dans le build.gradle

```
plugins {
    id 'java'
    id("org.hibernate.orm") version "8.0.0.Beta2"
}
```

JPA : Configuration Hibernate (session)

Dans le répertoire ressources hibernate.cfg.xml

```
<?xml version='1.0' encoding='utf-8'?>
<!DOCTYPE hibernate-configuration PUBLIC
        "-//Hibernate/Hibernate Configuration DTD//EN"
        "http://www.hibernate.org/dtd/hibernate-configuration-
3.0.dtd">
<hibernate-configuration>

    <session-factory>
        <property
name="connection.url">jdbc:h2:~/test</property>
        <property
name="connection.driver_class">org.h2.Driver</property>
        <property
name="hibernate.dialect">org.hibernate.dialect.H2Dialect
</property>
        <property name="connection.username">sa</property>
        <property name="connection.password"></property>
        <property name="show_sql">true</property>
        <property name="format_sql">true</property>
        <!-- DB schema will be updated if needed -->
        <property
name="hibernate.hbm2ddl.auto">create</property>
        <mapping class="iut.nantes.hibernate.entities.Product"/>
    </session-factory>
</hibernate-configuration>
```

JPA : Configuration Hibernate (session)

name	value
connection.url	jdbc:h2:~/test
connection.driver_class	org.h2.Driver
hibernate.dialect	org.hibernate.dialect.H2Dialect
connection.username	
connection.password	
…	…
hibernate.hbm2ddl.auto	create

<mapping class="iut.nantes.hibernate.entities.Product"/>

JPA : Configuration Hibernate (persistence unit)

```
<?xml version="1.0" encoding="UTF-8"?>
<persistence xmlns="https://jakarta.ee/xml/ns/persistence"
         version="3.0">

   <persistence-unit name="myPersistenceUnit">
      <provider>org.hibernate.jpa.HibernatePersistenceProvider</provider>

      <class>iut.nantes.hibernate.entities.Product</class>
      <properties>
         <property name="jakarta.persistence.jdbc.driver"
               value="org.h2.Driver"/>
         <property name="hibernate.dialect" value="org.hibernate.dialect.H2Dialect"/>
         <!-- Travail sur disque
         <property name="jakarta.persistence.jdbc.url"
               value="jdbc:h2:~/test"/>
               -->
         <property name="jakarta.persistence.jdbc.url"
               value="jdbc:h2:mem:test"/>
         <property name="jakarta.persistence.jdbc.user"
               value="sa"/>
         <property name="jakarta.persistence.jdbc.password"
               value=""/>

         <property name="hibernate.hbm2ddl.auto"
               value="update"/>
         <property name="hibernate.show_sql"
               value="true"/>
         <property name="show_sql" value="true" />
         <property name="format_sql" value="true" />
      </properties>

   </persistence-unit>
</persistence>
```

Dans ressources, META-INF, persistence.xml

JPA deux approches

Création du modèle de données depuis JPA

Utilisation du modèle et des données existantes

JPA Entité

Une **entité JPA** est une classe Java persistante dont les instances sont mappées à des lignes dans une table de base de données relationnelle.

Utilisation d’annotations

@Entity **public class Student** { // fields, getters and setters }

//Choix du nom de la table

@Entity
@Table(name="STUDENT") **public class Student** { // fields, getters and setters }

JPA Entité : @Id

Chaque entité devra avoir une clef primaire @Id

```
@Entity public class Student {
@Id
@GeneratedValue(strategy=GenerationType.AUTO)
private Long id;
```

JPA Entité : @Column

Possibilité de choisir la colonne et d’exprimer des contraintes d’intégrité

```
@Column(name="STUDENT_NAME", length=50, nullable=false, unique=false)
private String name;
```

JPA Entité : @Transient

Ne pas rendre persistant

```
@Transient
private Integer age;
```

JPA Entité : @Temporal

@Temporal est utilisée pour définir le type de stockage en base de données d'un champ de type java.util.Date ou java.util.Calendar.

```
// Stockera uniquement le jour, le mois et l'année (ex: Date de naissance)
@Temporal(TemporalType.DATE) private Date birthDate;

// Stockera uniquement l'heure (ex: Début de service)
@Temporal(TemporalType.TIME) private Date shiftStartTime;

// Stockera la date complète et l'heure exacte (ex: Date de création du compte)
@Temporal(TemporalType.TIMESTAMP) private Date createdAt;
```

JPA Entité : @Enumerated

@Enumerated permet de définir la manière dont un type énuméré (enum) doit être stocké en base de données

public enum StatutContrat { STAGE, // Ordinal 0

CDD, // Ordinal 1

CDI // Ordinal 2 }

// Sera enregistré sous forme de chaîne (ex: "CDI") dans la colonne
@Enumerated(EnumType.STRING)
private StatutContrat statut;

‡// Sans spécification, sera enregistré sous forme d'index (ex: 1 pour CDD)
@Enumerated(EnumType.ORDINAL)

private StatutContrat typeDefaut;

JPA Entity

```
@Entity
@Table(name="STUDENT")
public class Student {
@Id
@GeneratedValue(strategy=GenerationType.AUTO)
private Long id;
@Column(name="STUDENT_NAME", length=50, nullable=false, unique=false)
private String name;
@Transient
private Integer age;
@Temporal(TemporalType.DATE)
private Date birthDate;
@Enumerated(EnumType.STRING)
private Gender gender;
// other fields, getters and setters }
```

JPA Cardinalités

- 1-1 OneToOne rem : n’a de sens qu’en POO pas en relationnel
- n-1 ManyToOne
- 1-n OneToMany
- n-n ManyToMany

Les n-n with parameter ne sont pas possibles directement

La navigabilité peut-être exprimée.

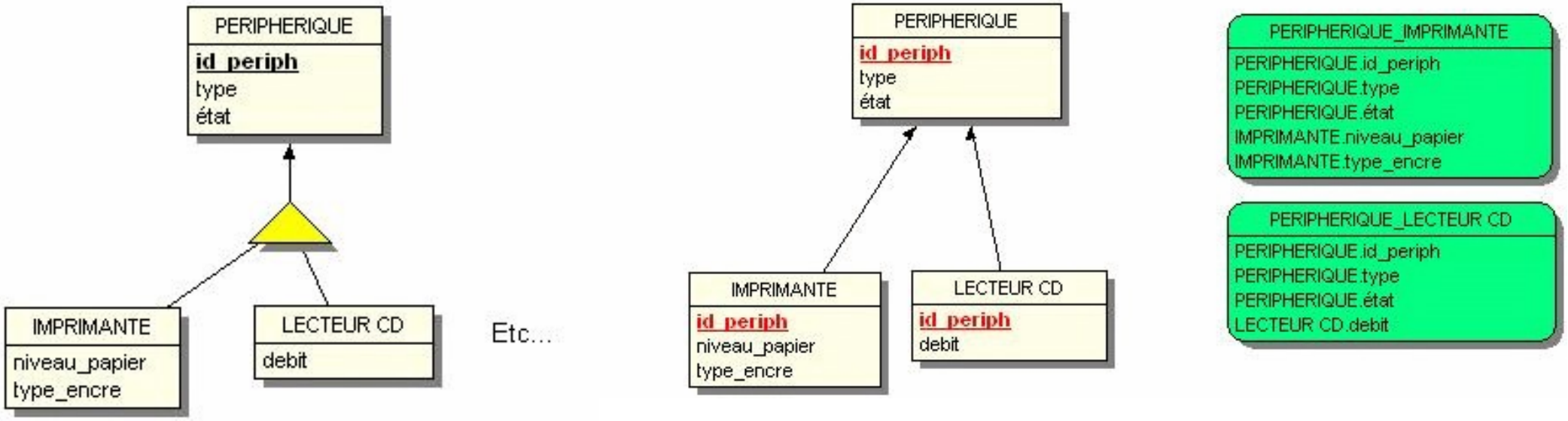
JPA Cardinalité : OneToOne

```
@Entity
public class Utilisateur {
@Id
@GeneratedValue(strategy = GenerationType.IDENTITY)
private Long id;
private String nom; @OneToOne(cascade = CascadeType.ALL)
@JoinColumn(name = "adresse_id", referencedColumnName = "id")
private Adresse adresse;
}
```

```
@Entity public class Adresse {
@Id
@GeneratedValue(strategy = GenerationType.IDENTITY)
private Long id;
private String rue;
@OneToOne(mappedBy = "adresse")
// Doit correspondre au nom du champ dans Utilisateur
private Utilisateur utilisateur; }
```

JPA Cardinalité : OneToOne

Peut-être utilisée pour simuler un héritage avec un partage de la clef primaire



https://merise.developpez.com/faq/preview/?page=MLD

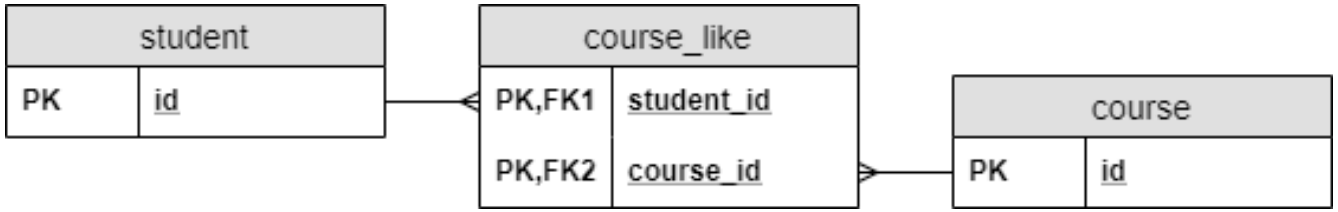
JPA Cardinalité : OneToMany

```
@Entity
public class Department {
@Id
@GeneratedValue(strategy = GenerationType.IDENTITY)
private Long id;
private String name; @OneToMany(mappedBy = "department", cascade = CascadeType.ALL, orphanRemoval = true)
private List<Employee> employees = new ArrayList<>();
// Getters et Setters
}
```

```
@Entity public class Employee {
@Id
@GeneratedValue(strategy = GenerationType.IDENTITY)
private Long id;
private String name;
@ManyToOne(fetch = FetchType.LAZY) @JoinColumn(name = "department_id") private Department department;
// Getters et Setters
}
```

JPA Cardinalité : ManyToMany





JPA Cardinalité : ManyToMany

```
@Entity
class Student {
@Id Long id;
@ManyToMany Set<Course>
likedCourses;
// additional properties
// standard constructors, getters,
and setters }
```

```
@Entity
class Course {
@Id Long id;
@ManyToMany Set<Student>
likes;
// additional properties
// standard constructors, getters,
and setters }
```

JPA Cardinalité : ManyToMany

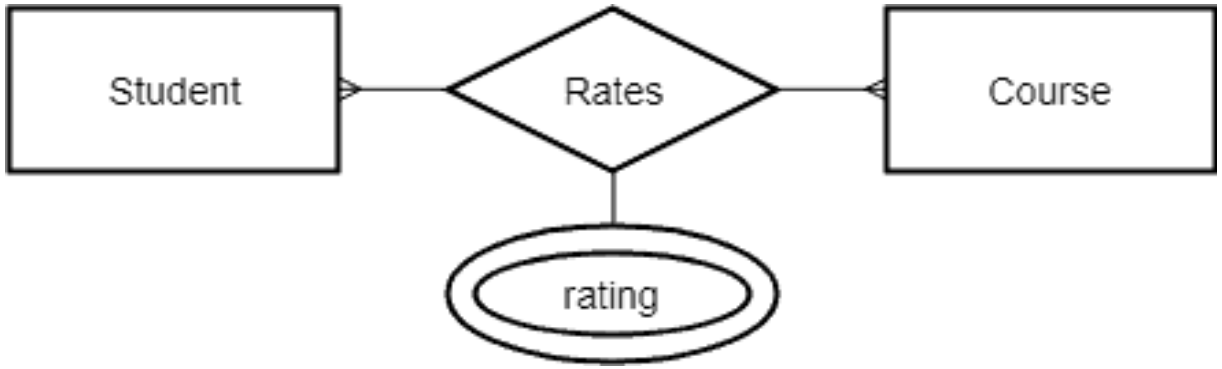
Affiner la jointure

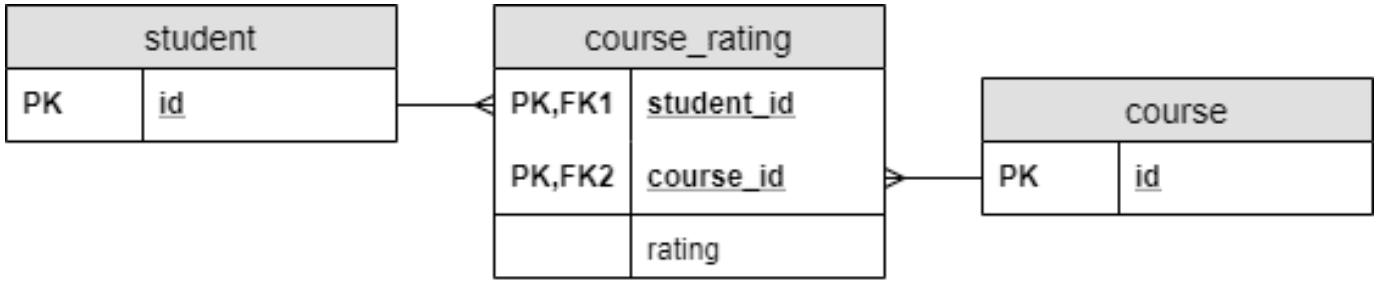
@ManyToMany @JoinTable(name = "course_like", joinColumns = @JoinColumn(name = "student_id"), inverseJoinColumns = @JoinColumn(name = "course_id")) Set<Course> likedCourses;

//Coté cible

@ManyToMany(mappedBy = "likedCourses") Set<Student> likes;

JPA Cardinalité : ManyToMany with parameter





JPA Cardinalité : ManyToMany with parameter

Création d’une entité encapsulée

Clef primaire composée des 2 autres

Rem : @Embeddable

Classe peut-être interne

```
@Embeddable
class CourseRatingKey implements Serializable { @Column(name = "student_id")
Long studentId;
@Column(name = "course_id")
Long courseId;
// standard constructors, getters, and setters
// hashcode and equals implementation }
```

JPA Cardinalité : ManyToMany with parameter

Utilisation de l’entité avec la clef composite pour définir la jointure

```
@Entity class CourseRating {
@EmbeddedId CourseRatingKey id;
@ManyToOne @MapsId("studentId")
@JoinColumn(name = "student_id")
Student student;
@ManyToOne
@MapsId("courseId")
@JoinColumn(name = "course_id")
Course course;
int rating;
// standard constructors, getters, and setters }
```

JPA : Lazy vs Eager

La différence principale entre Lazy et Eager réside dans **le moment où les données associées (relations) sont chargées depuis la base de données**. Le mode LAZY (chargement paresseux) diffère le chargement jusqu'à ce que la relation soit explicitement accédée dans le code, tandis que le mode EAGER (chargement immédiat) récupère tout en une seule fois en même temps que l'entité principale.

JPA : Entity Manager

L'EntityManager fait office de passerelle entre votre code orienté objet et le modèle relationnel de la base de données. Ses principales responsabilités incluent :

- La gestion du cycle de vie des objets : Suivre l'état des entités (nouvelle, gérée, détachée, supprimée).
- Les opérations CRUD : Sauvegarder, lire, mettre à jour et supprimer des données.
- La gestion des requêtes : Créer et exécuter des requêtes dynamiques ou nommées en JPQL ou en SQL natif.
- La synchronisation : Valider et répercuter les changements en mémoire vers la base de données (lors d'un commit ou d'un flush).

JPA : Entity Manager

```
EntityManagerFactory emf =
        Persistence.createEntityManagerFactory("myPersistenceUnit");
EntityManager em = emf.createEntityManager();


em.getTransaction().begin();

…

em.getTransaction().commit();
```

JPA : Entity Manager persitence

```
Product product1 = new Product("Test1",10.0,2);
Product product2 = new Product("Test2",220.0,20);
//Persistance
em.persist(product1);
em.persist(product2);
```

JPA : Entity Manager recherche

Par clef primaire

```
Product p = em.find( Product.class, 123 );
```

Par requête

```
List<Product> products;
//Requête
TypedQuery<Product> query = em.createQuery("from Product",
Product.class);
products= query.getResultList();
```

JPA : Entity Manager mise à jour

…

```
product1.setPrice(1000.5);
em.persist(product1);
```

JPA : Entity Manager suppression

```
em.remove(product1);
```

JPA : Repository